

Стохастический градиентный спуск

Семинар

ФКН ВШЭ

Задача с конечной суммой

Рассмотрим классическую задачу минимизации среднего по конечной выборке:

$$\min_{x \in \mathbb{R}^p} f(x) = \min_{x \in \mathbb{R}^p} \frac{1}{n} \sum_{i=1}^n f_i(x)$$

Градиентный спуск выполняет следующие обновления:

$$x_{k+1} = x_k - \frac{\alpha_k}{n} \sum_{i=1}^n \nabla f_i(x) \quad (\text{GD})$$

- Сходимость при постоянном α или при линейном поиске.

Задача с конечной суммой

Рассмотрим классическую задачу минимизации среднего по конечной выборке:

$$\min_{x \in \mathbb{R}^p} f(x) = \min_{x \in \mathbb{R}^p} \frac{1}{n} \sum_{i=1}^n f_i(x)$$

Градиентный спуск выполняет следующие обновления:

$$x_{k+1} = x_k - \frac{\alpha_k}{n} \sum_{i=1}^n \nabla f_i(x) \quad (\text{GD})$$

- Сходимость при постоянном α или при линейном поиске.
- Стоимость итерации линейна по n . Для ImageNet $n \approx 1.4 \cdot 10^7$, для WikiText $n \approx 10^8$.

Задача с конечной суммой

Рассмотрим классическую задачу минимизации среднего по конечной выборке:

$$\min_{x \in \mathbb{R}^p} f(x) = \min_{x \in \mathbb{R}^p} \frac{1}{n} \sum_{i=1}^n f_i(x)$$

Градиентный спуск выполняет следующие обновления:

$$x_{k+1} = x_k - \frac{\alpha_k}{n} \sum_{i=1}^n \nabla f_i(x) \quad (\text{GD})$$

- Сходимость при постоянном α или при линейном поиске.
- Стоимость итерации линейна по n . Для ImageNet $n \approx 1.4 \cdot 10^7$, для WikiText $n \approx 10^8$.

Задача с конечной суммой

Рассмотрим классическую задачу минимизации среднего по конечной выборке:

$$\min_{x \in \mathbb{R}^p} f(x) = \min_{x \in \mathbb{R}^p} \frac{1}{n} \sum_{i=1}^n f_i(x)$$

Градиентный спуск выполняет следующие обновления:

$$x_{k+1} = x_k - \frac{\alpha_k}{n} \sum_{i=1}^n \nabla f_i(x) \quad (\text{GD})$$

- Сходимость при постоянном α или при линейном поиске.
- Стоимость итерации линейна по n . Для ImageNet $n \approx 1.4 \cdot 10^7$, для WikiText $n \approx 10^8$.

Заменим вычисление полного градиента на его несмещённую оценку, случайно выбирая индекс i_k точки на каждой итерации равномерно:

$$x_{k+1} = x_k - \alpha_k \nabla f_{i_k}(x_k) \quad (\text{SGD})$$

При $p(i_k = i) = \frac{1}{n}$ стохастический градиент является несмещённой оценкой градиента:

$$\mathbb{E}[\nabla f_{i_k}(x)] = \sum_{i=1}^n p(i_k = i) \nabla f_i(x) = \sum_{i=1}^n \frac{1}{n} \nabla f_i(x) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(x) = \nabla f(x)$$

Это означает, что математическое ожидание стохастического градиента равно истинному градиенту $f(x)$.

Результаты для градиентного спуска

Стохастические итерации в n раз дешевле, но сколько итераций требуется?

Если ∇f липшицево непрерывен, то:

Предположение	Детерминированный градиентный спуск	Стохастический градиентный спуск
PL	$O(\log(1/\varepsilon))$	$O(1/\varepsilon)$
Выпуклый	$O(1/\varepsilon)$	$O(1/\varepsilon^2)$
Невыпуклый	$O(1/\varepsilon)$	$O(1/\varepsilon^2)$

- Стохастический метод имеет низкую стоимость итерации, но медленную скорость сходимости.

Результаты для градиентного спуска

Стохастические итерации в n раз дешевле, но сколько итераций требуется?

Если ∇f липшицево непрерывен, то:

Предположение	Детерминированный градиентный спуск	Стохастический градиентный спуск
PL	$O(\log(1/\varepsilon))$	$O(1/\varepsilon)$
Выпуклый	$O(1/\varepsilon)$	$O(1/\varepsilon^2)$
Невыпуклый	$O(1/\varepsilon)$	$O(1/\varepsilon^2)$

- Стохастический метод имеет низкую стоимость итерации, но медленную скорость сходимости.
 - Сублинейная скорость даже в сильно выпуклом случае.

Результаты для градиентного спуска

Стохастические итерации в n раз дешевле, но сколько итераций требуется?

Если ∇f липшицево непрерывен, то:

Предположение	Детерминированный градиентный спуск	Стохастический градиентный спуск
PL	$O(\log(1/\varepsilon))$	$O(1/\varepsilon)$
Выпуклый	$O(1/\varepsilon)$	$O(1/\varepsilon^2)$
Невыпуклый	$O(1/\varepsilon)$	$O(1/\varepsilon^2)$

- Стохастический метод имеет низкую стоимость итерации, но медленную скорость сходимости.
 - Сублинейная скорость даже в сильно выпуклом случае.
 - Оценки неулучшаемы при стандартных предположениях.

Результаты для градиентного спуска

Стохастические итерации в n раз дешевле, но сколько итераций требуется?

Если ∇f липшицево непрерывен, то:

Предположение	Детерминированный градиентный спуск	Стохастический градиентный спуск
PL	$O(\log(1/\varepsilon))$	$O(1/\varepsilon)$
Выпуклый	$O(1/\varepsilon)$	$O(1/\varepsilon^2)$
Невыпуклый	$O(1/\varepsilon)$	$O(1/\varepsilon^2)$

- Стохастический метод имеет низкую стоимость итерации, но медленную скорость сходимости.
 - Сублинейная скорость даже в сильно выпуклом случае.
 - Оценки неулучшаемы при стандартных предположениях.
 - Оракул возвращает несмещённое приближение градиента с ограниченной дисперсией.

Результаты для градиентного спуска

Стохастические итерации в n раз дешевле, но сколько итераций требуется?

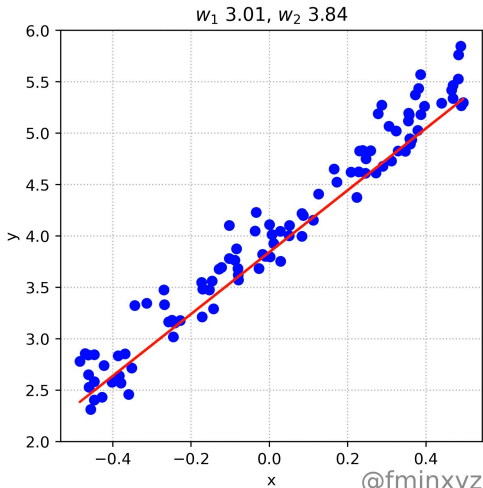
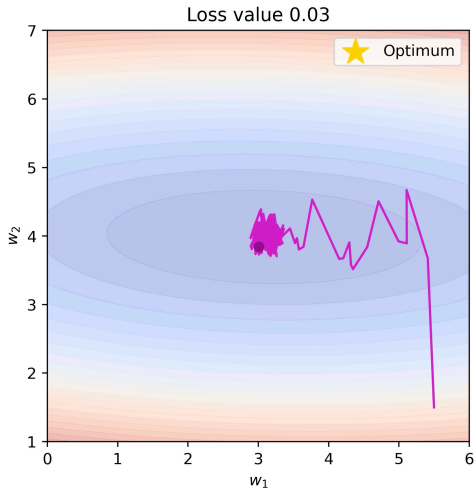
Если ∇f липшицево непрерывен, то:

Предположение	Детерминированный градиентный спуск	Стохастический градиентный спуск
PL	$O(\log(1/\varepsilon))$	$O(1/\varepsilon)$
Выпуклый	$O(1/\varepsilon)$	$O(1/\varepsilon^2)$
Невыпуклый	$O(1/\varepsilon)$	$O(1/\varepsilon^2)$

- Стохастический метод имеет низкую стоимость итерации, но медленную скорость сходимости.
 - Сублинейная скорость даже в сильно выпуклом случае.
 - Оценки неулучшаемы при стандартных предположениях.
 - Оракул возвращает несмещённое приближение градиента с ограниченной дисперсией.
- Методы с моментом и квазиньютоновские методы не улучшают скорость сходимости в стохастическом случае. Могут улучшить лишь константные факторы (узкое место — дисперсия, а не число обусловленности).

Типичное поведение

Stochastic Gradient Descent. Batch = 2



Вычислительные эксперименты

Визуализация SGD.

Посмотрим на вычислительные эксперименты для SGD .

AdaGrad (Duchi, Hazan, and Singer 2010)

Очень популярный адаптивный метод. Пусть $g^{(k)} = \nabla f_{i_k}(x^{(k-1)})$, обновление для $j = 1, \dots, p$:

$$v_j^{(k)} = v_j^{(k-1)} + (g_j^{(k)})^2$$
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \epsilon}}$$

Примечания:

- AdaGrad не требует подбора скорости обучения: $\alpha > 0$ — фиксированная константа, скорость обучения естественно убывает по итерациям.

AdaGrad (Duchi, Hazan, and Singer 2010)

Очень популярный адаптивный метод. Пусть $g^{(k)} = \nabla f_{i_k}(x^{(k-1)})$, обновление для $j = 1, \dots, p$:

$$v_j^{(k)} = v_j^{(k-1)} + (g_j^{(k)})^2$$
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \epsilon}}$$

Примечания:

- AdaGrad не требует подбора скорости обучения: $\alpha > 0$ — фиксированная константа, скорость обучения естественно убывает по итерациям.
- Скорость обучения редких информативных признаков убывает медленно.

AdaGrad (Duchi, Hazan, and Singer 2010)

Очень популярный адаптивный метод. Пусть $g^{(k)} = \nabla f_{i_k}(x^{(k-1)})$, обновление для $j = 1, \dots, p$:

$$v_j^{(k)} = v_j^{(k-1)} + (g_j^{(k)})^2$$
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \epsilon}}$$

Примечания:

- AdaGrad не требует подбора скорости обучения: $\alpha > 0$ — фиксированная константа, скорость обучения естественно убывает по итерациям.
- Скорость обучения редких информативных признаков убывает медленно.
- Может значительно превосходить SGD на разреженных задачах.

AdaGrad (Duchi, Hazan, and Singer 2010)

Очень популярный адаптивный метод. Пусть $g^{(k)} = \nabla f_{i_k}(x^{(k-1)})$, обновление для $j = 1, \dots, p$:

$$v_j^{(k)} = v_j^{(k-1)} + (g_j^{(k)})^2$$
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \epsilon}}$$

Примечания:

- AdaGrad не требует подбора скорости обучения: $\alpha > 0$ — фиксированная константа, скорость обучения естественно убывает по итерациям.
- Скорость обучения редких информативных признаков убывает медленно.
- Может значительно превосходить SGD на разреженных задачах.
- Главный недостаток — монотонное накопление градиентов в знаменателе. Adadelata, Adam, AMSGrad и другие методы устраняют этот недостаток и популярны при обучении глубоких нейронных сетей.

AdaGrad (Duchi, Hazan, and Singer 2010)

Очень популярный адаптивный метод. Пусть $g^{(k)} = \nabla f_{i_k}(x^{(k-1)})$, обновление для $j = 1, \dots, p$:

$$v_j^{(k)} = v_j^{(k-1)} + (g_j^{(k)})^2$$
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \epsilon}}$$

Примечания:

- AdaGrad не требует подбора скорости обучения: $\alpha > 0$ — фиксированная константа, скорость обучения естественно убывает по итерациям.
- Скорость обучения редких информативных признаков убывает медленно.
- Может значительно превосходить SGD на разреженных задачах.
- Главный недостаток — монотонное накопление градиентов в знаменателе. Adadelata, Adam, AMSGrad и другие методы устраняют этот недостаток и популярны при обучении глубоких нейронных сетей.
- Константа ϵ обычно задаётся равной 10^{-6} , чтобы избежать деления на ноль или слишком больших размеров шага.

RMSPProp (Tieleman and Hinton, 2012)

Улучшение AdaGrad, устраняющее его агрессивно монотонно убывающую скорость обучения. Использует скользящее среднее квадратов градиентов для адаптации скорости обучения для каждого веса. Пусть $g^{(k)} = \nabla f_{i_k}(x^{(k-1)})$, правило обновления для $j = 1, \dots, p$:

$$v_j^{(k)} = \gamma v_j^{(k-1)} + (1 - \gamma)(g_j^{(k)})^2$$

$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \epsilon}}$$

Примечания:

- RMSPProp делит скорость обучения для веса на скользящее среднее магнитуд недавних градиентов по этому весу.

RMSPProp (Tieleman and Hinton, 2012)

Улучшение AdaGrad, устраняющее его агрессивно монотонно убывающую скорость обучения. Использует скользящее среднее квадратов градиентов для адаптации скорости обучения для каждого веса. Пусть $g^{(k)} = \nabla f_{i_k}(x^{(k-1)})$, правило обновления для $j = 1, \dots, p$:

$$v_j^{(k)} = \gamma v_j^{(k-1)} + (1 - \gamma)(g_j^{(k)})^2$$

$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \epsilon}}$$

Примечания:

- RMSPProp делит скорость обучения для веса на скользящее среднее магнитуд недавних градиентов по этому весу.
- Обеспечивает более тонкую адаптацию скоростей обучения по сравнению с AdaGrad, что делает метод подходящим для нестационарных задач.

RMSPProp (Tieleman and Hinton, 2012)

Улучшение AdaGrad, устраняющее его агрессивно монотонно убывающую скорость обучения. Использует скользящее среднее квадратов градиентов для адаптации скорости обучения для каждого веса. Пусть $g^{(k)} = \nabla f_{i_k}(x^{(k-1)})$, правило обновления для $j = 1, \dots, p$:

$$v_j^{(k)} = \gamma v_j^{(k-1)} + (1 - \gamma)(g_j^{(k)})^2$$

$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{g_j^{(k)}}{\sqrt{v_j^{(k)} + \epsilon}}$$

Примечания:

- RMSPProp делит скорость обучения для веса на скользящее среднее магнитуд недавних градиентов по этому весу.
- Обеспечивает более тонкую адаптацию скоростей обучения по сравнению с AdaGrad, что делает метод подходящим для нестационарных задач.
- Широко используется при обучении нейронных сетей, особенно рекуррентных.

Adadelta (Zeiler, 2012)

Расширение RMSProp, уменьшающее зависимость от вручную задаваемой глобальной скорости обучения. Вместо накопления всех прошлых квадратов градиентов, Adadelta ограничивает окно накопленных прошлых градиентов некоторым фиксированным размером w . Механизм обновления не требует скорости обучения α :

$$\begin{aligned}v_j^{(k)} &= \gamma v_j^{(k-1)} + (1 - \gamma)(g_j^{(k)})^2 \\ \tilde{g}_j^{(k)} &= \frac{\sqrt{\Delta x_j^{(k-1)} + \epsilon}}{\sqrt{v_j^{(k)} + \epsilon}} g_j^{(k)} \\ x_j^{(k)} &= x_j^{(k-1)} - \tilde{g}_j^{(k)} \\ \Delta x_j^{(k)} &= \rho \Delta x_j^{(k-1)} + (1 - \rho)(\tilde{g}_j^{(k)})^2\end{aligned}$$

Примечания:

- Adadelta адаптирует скорости обучения на основе скользящего окна обновлений градиента, а не накапливает все прошлые градиенты. Благодаря этому адаптированные скорости обучения более устойчивы к изменениям динамики модели.

Adadelta (Zeiler, 2012)

Расширение RMSProp, уменьшающее зависимость от вручную задаваемой глобальной скорости обучения. Вместо накопления всех прошлых квадратов градиентов, Adadelta ограничивает окно накопленных прошлых градиентов некоторым фиксированным размером w . Механизм обновления не требует скорости обучения α :

$$\begin{aligned}v_j^{(k)} &= \gamma v_j^{(k-1)} + (1 - \gamma)(g_j^{(k)})^2 \\ \tilde{g}_j^{(k)} &= \frac{\sqrt{\Delta x_j^{(k-1)} + \epsilon}}{\sqrt{v_j^{(k)} + \epsilon}} g_j^{(k)} \\ x_j^{(k)} &= x_j^{(k-1)} - \tilde{g}_j^{(k)} \\ \Delta x_j^{(k)} &= \rho \Delta x_j^{(k-1)} + (1 - \rho)(\tilde{g}_j^{(k)})^2\end{aligned}$$

Примечания:

- Adadelta адаптирует скорости обучения на основе скользящего окна обновлений градиента, а не накапливает все прошлые градиенты. Благодаря этому адаптированные скорости обучения более устойчивы к изменениям динамики модели.
- Метод не требует задания начальной скорости обучения, что упрощает настройку.

Adadelata (Zeiler, 2012)

Расширение RMSProp, уменьшающее зависимость от вручную задаваемой глобальной скорости обучения. Вместо накопления всех прошлых квадратов градиентов, Adadelata ограничивает окно накопленных прошлых градиентов некоторым фиксированным размером w . Механизм обновления не требует скорости обучения α :

$$\begin{aligned}v_j^{(k)} &= \gamma v_j^{(k-1)} + (1 - \gamma)(g_j^{(k)})^2 \\ \tilde{g}_j^{(k)} &= \frac{\sqrt{\Delta x_j^{(k-1)} + \epsilon}}{\sqrt{v_j^{(k)} + \epsilon}} g_j^{(k)} \\ x_j^{(k)} &= x_j^{(k-1)} - \tilde{g}_j^{(k)} \\ \Delta x_j^{(k)} &= \rho \Delta x_j^{(k-1)} + (1 - \rho)(\tilde{g}_j^{(k)})^2\end{aligned}$$

Примечания:

- Adadelata адаптирует скорости обучения на основе скользящего окна обновлений градиента, а не накапливает все прошлые градиенты. Благодаря этому адаптированные скорости обучения более устойчивы к изменениям динамики модели.
- Метод не требует задания начальной скорости обучения, что упрощает настройку.
- Часто применяется в глубоком обучении, где масштабы параметров существенно различаются между слоями.

Adam (Kingma and Ba, 2014)^{1 2}

Объединяет элементы AdaGrad и RMSProp. Использует экспоненциально затухающее среднее прошлых градиентов и квадратов градиентов.

EMA:
$$m_j^{(k)} = \beta_1 m_j^{(k-1)} + (1 - \beta_1) g_j^{(k)}$$

Коррекция смещения:
$$\hat{m}_j = \frac{m_j^{(k)}}{1 - \beta_1^k}$$

Обновление:
$$x_j^{(k)} = x_j^{(k-1)} - \alpha \frac{\hat{m}_j}{\sqrt{\hat{v}_j} + \epsilon}$$

$$v_j^{(k)} = \beta_2 v_j^{(k-1)} + (1 - \beta_2) (g_j^{(k)})^2$$

$$\hat{v}_j = \frac{v_j^{(k)}}{1 - \beta_2^k}$$

Примечания: * Метод корректирует смещение начальных моментов к нулю, характерное для других методов вроде RMSProp, что делает оценки более точными. * Одна из наиболее цитируемых научных статей в мире. * В 2018–2019 годах были опубликованы статьи, указывающие на ошибки в оригинальной работе. * Не сходится на некоторых простых задачах (даже выпуклых). * Тем не менее исключительно хорошо работает на ряде сложных задач. * Значительно лучше работает для языковых моделей, чем для задач компьютерного зрения — почему?

¹Adam: A Method for Stochastic Optimization

²On the Convergence of Adam and Beyond

AdamW (Loshchilov & Hutter, 2017)

Устраняет распространённую проблему с ℓ_2 -регуляризацией в адаптивных оптимизаторах вроде Adam.

Стандартная ℓ_2 -регуляризация добавляет $\lambda\|x\|^2$ к функции потерь, что порождает слагаемое градиента λx . В

Adam это слагаемое масштабируется адаптивной скоростью обучения $\left(\sqrt{\hat{v}_j} + \epsilon\right)$, связывая затухание весов с магнитудами градиентов.

AdamW разделяет затухание весов и шаг адаптации градиента.

Правило обновления:

$$\begin{aligned}m_j^{(k)} &= \beta_1 m_j^{(k-1)} + (1 - \beta_1) g_j^{(k)} \\v_j^{(k)} &= \beta_2 v_j^{(k-1)} + (1 - \beta_2) (g_j^{(k)})^2 \\ \hat{m}_j &= \frac{m_j^{(k)}}{1 - \beta_1^k}, \quad \hat{v}_j = \frac{v_j^{(k)}}{1 - \beta_2^k} \\ x_j^{(k)} &= x_j^{(k-1)} - \alpha \left(\frac{\hat{m}_j}{\sqrt{\hat{v}_j} + \epsilon} + \lambda x_j^{(k-1)} \right)\end{aligned}$$

Примечания:

- Слагаемое затухания весов $\lambda x_j^{(k-1)}$ добавляется *после* шага адаптивного градиента.

AdamW (Loshchilov & Hutter, 2017)

Устраняет распространённую проблему с ℓ_2 -регуляризацией в адаптивных оптимизаторах вроде Adam.

Стандартная ℓ_2 -регуляризация добавляет $\lambda\|x\|^2$ к функции потерь, что порождает слагаемое градиента λx . В

Adam это слагаемое масштабируется адаптивной скоростью обучения $(\sqrt{\hat{v}_j} + \epsilon)$, связывая затухание весов с магнитудами градиентов.

AdamW разделяет затухание весов и шаг адаптации градиента.

Правило обновления:

$$\begin{aligned}m_j^{(k)} &= \beta_1 m_j^{(k-1)} + (1 - \beta_1) g_j^{(k)} \\v_j^{(k)} &= \beta_2 v_j^{(k-1)} + (1 - \beta_2) (g_j^{(k)})^2 \\ \hat{m}_j &= \frac{m_j^{(k)}}{1 - \beta_1^k}, \quad \hat{v}_j = \frac{v_j^{(k)}}{1 - \beta_2^k} \\ x_j^{(k)} &= x_j^{(k-1)} - \alpha \left(\frac{\hat{m}_j}{\sqrt{\hat{v}_j} + \epsilon} + \lambda x_j^{(k-1)} \right)\end{aligned}$$

Примечания:

- Слагаемое затухания весов $\lambda x_j^{(k-1)}$ добавляется *после* шага адаптивного градиента.
- Широко применяется при обучении трансформеров и других больших моделей. Выбор по умолчанию для